SQL SERVER TO FABRIC

End-to-End Migration & Development Guide

SQL Server → Lakehouse → Semantic Model → Power BI

Mapped to JD: Fabric Migration & Curated Layer Development

Prepared by Sunday Dada | March 2026

Overview

This guide walks through the complete migration path from SQL Server / Power BI Desktop to Microsoft Fabric, then through building curated semantic models to Power BI report delivery. It follows the exact sequence described in the job description, organized into 7 phases:

Phase	Name	JD Requirement	Output
1	Discovery & Audit	Lead migration of existing BI datasets	Migration inventory
2	Fabric Environment Setup	Ensure Fabric supports scalable reporting	Workspaces, capacity, Git
3	Data Ingestion to Lakehouse	Migrate from SQL Server to Fabric	Bronze/Silver tables
4	Curated Layer Development	Build curated layer with KPI logic	Gold tables and views
5	Semantic Model Development	Scalable Power BI semantic models	Direct Lake datasets
6	Power BI Report Development	Executive reporting, dashboards	Reports + Power BI App
7	Governance & Documentation	Best practices, documented datasets	Governance framework

Phase 1: Discover and Audit the Existing Environment

JD: *"Lead migration of existing BI datasets from SQL Server / Power BI Desktop to Microsoft Fabric"*

Before migrating anything, you need a complete picture of what exists. This phase produces the migration inventory.

Step 1.1: Audit the SQL Server Environment

- Connect to the existing SQL Server instance(s) and catalog every database, schema, table, and view that feeds BI reporting
- Document for each table: row count, data size, last modified date, refresh frequency, and any dependencies (stored procedures, triggers, linked servers)
- Identify which tables are actively used in Power BI reports vs. legacy/orphaned tables

```
-- Run this on each SQL Server database to get table inventory
SELECT s.name AS SchemaName, t.name AS TableName,
  p.rows AS RowCount,
  SUM(a.total_pages) * 8 / 1024 AS SizeMB
FROM sys.tables t
  JOIN sys.schemas s ON t.schema_id = s.schema_id
  JOIN sys.partitions p ON t.object_id = p.object_id
  JOIN sys.allocation_units a ON p.partition_id = a.container_id
WHERE p.index_id IN (0,1)
GROUP BY s.name, t.name, p.rows
ORDER BY SizeMB DESC
```

Step 1.2: Audit the Power BI Desktop Environment

- Collect all .pbix files currently in use (production reports)
- For each .pbix file, document: data sources (SQL Server queries, Excel, etc.), Import/DirectQuery mode, dataset size, refresh schedule, number of DAX measures, and consuming users/groups
- Identify shared datasets vs. report-specific datasets
- Flag any Power BI Gen1 Dataflows that exist in the current Power BI Service tenant
- Use the Power BI REST API or third-party scanner (e.g., ALM Toolkit, Tabular Editor) to extract dataset metadata programmatically

Step 1.3: Identify Existing Gen1 Dataflows

- In the Power BI Service, go to each workspace and check for Gen1 Dataflows
- Document each Dataflow: source connections, transformations applied, output datasets, refresh schedule
- These will be migrated to Dataflow Gen2 in Phase 3

Step 1.4: Map Dependencies and Build Migration Inventory

Create a migration inventory spreadsheet with these columns:

- **Source Object** (SQL table name, .pbix file name, Dataflow name)
- **Source System** (SQL Server instance, Power BI workspace)
- **Data Volume** (row count, size in MB)
- **Downstream Consumers** (which reports/dashboards use this data)
- **Target Layer** (Bronze, Silver, or Gold)
- **Migration Method** (Data Pipeline CopyData, Dataflow Gen2, Database Mirroring)
- **Priority** (High/Medium/Low based on business criticality)
- **Estimated Effort** (hours)

Step 1.5: Determine SQL Server Connectivity

Critical question: is the SQL Server Azure SQL or on-premises?

- **Azure SQL Database:** You can use Database Mirroring (preview) for near-real-time replication into Fabric, OR Data Pipeline CopyData for batch loads. No gateway needed.
- **On-premises SQL Server:** You need an On-Premises Data Gateway installed and configured. Data Pipeline CopyData and Dataflow Gen2 both support gateway connections.
- Document the connection details: server name, authentication method (SQL auth, Windows auth, Azure AD), firewall rules, and any VPN requirements

✓ CHECKPOINT: You should now have a complete migration inventory with every SQL table, .pbix file, and Dataflow cataloged, prioritized, and assigned a migration method.

Phase 2: Set Up the Fabric Environment

JD: *"Ensure the Fabric environment supports scalable reporting, self-service analytics, and future AI capabilities"*

Step 2.1: Provision Fabric Capacity

- In the Azure Portal, create a Microsoft Fabric capacity
- Choose an SKU based on the data volumes identified in Phase 1 and expected concurrent users
- Assign the capacity to your Fabric tenant
- Enable Fabric in the Power BI Admin Portal (Admin Portal > Tenant Settings > Microsoft Fabric)

⚠ If heavy ETL workloads (Data Engineering) will run alongside interactive report queries, discuss separate capacities with IT to avoid compute contention.

Step 2.2: Create the Workspace Structure

Create these workspaces in Fabric, each assigned to the provisioned capacity:

- **WS-Bronze-DEV / WS-Bronze-PROD** — Raw data landing zone. Owned by Data Engineering.
- **WS-Silver-DEV / WS-Silver-PROD** — Cleansed and conformed data. Owned by Data Engineering, read access for BI Developers.
- **WS-Gold-Curated-DEV** — Your primary workspace. Build semantic models, Dataflows, KPI views here.
- **WS-Gold-Curated-TEST** — Stakeholder validation before production.
- **WS-Gold-Curated-PROD** — Production curated layer serving reports.
- **WS-Reporting-PROD** — Published Power BI reports and Power BI App. Consumer-facing.

Step 2.3: Create Security Groups

In Entra ID (Azure AD), create these security groups and assign to workspaces:

- **SG-Fabric-DataEngineers** → Admin on WS-Bronze, WS-Silver | No access to WS-Gold
- **SG-Fabric-BIDevelopers** → Contributor on WS-Gold-Curated-DEV/TEST/PROD | Viewer on WS-Silver
- **SG-Fabric-ReportConsumers** → Viewer on WS-Reporting-PROD only
- **SG-Fabric-SelfServiceAnalysts** → Viewer on WS-Gold-Curated-PROD with Build permission on semantic models

Step 2.4: Set Up Deployment Pipelines

- In Fabric, go to Deployment Pipelines and create a pipeline for the Gold-Curated layer
- Assign: DEV → TEST → PROD workspaces
- This lets you promote Dataflows, semantic models, and reports across environments with one click
- Also set up a separate pipeline for the Reporting workspace if reports are developed separately from datasets

Step 2.5: Connect DEV Workspace to Git

- Go to Workspace Settings > Git Integration
- Connect WS-Gold-Curated-DEV to an Azure DevOps or GitHub repository
- Select the branch (e.g., main or develop)
- Sync Fabric items (semantic models, notebooks, pipeline definitions) to the repository
- This provides version control, audit trail, and collaborative development for all curated layer items
- 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
-

Step 2.6: Create the Lakehouses

In each workspace, create the appropriate Lakehouse:

- **LH-Bronze** (in WS-Bronze) — raw data from SQL Server lands here as-is
- **LH-Silver** (in WS-Silver) — cleansed, deduplicated, conformed data
- **LH-Gold** (in WS-Gold-Curated) — business-ready tables for semantic models. THIS IS YOUR PRIMARY DELIVERABLE.

✓ CHECKPOINT: You should now have capacity provisioned, all workspaces created with security groups assigned, deployment pipelines configured, Git connected, and three Lakehouses (Bronze, Silver, Gold) created.

Phase 3: Migrate Data from SQL Server to Fabric Lakehouse

***JD:** "Lead migration of existing BI datasets from SQL Server / Power BI Desktop to Microsoft Fabric" + "Implement Dataflows"*

Step 3.1: Set Up Connectivity

- **For Azure SQL:** No gateway needed. Create a connection in Fabric (Settings > Manage Connections) using Azure SQL connector with Azure AD or SQL authentication.
- **For on-premises SQL Server:** Install and configure an On-Premises Data Gateway. Register it in the Fabric Admin Portal. Create a connection using the gateway.
- Test connectivity by creating a simple Dataflow Gen2 that queries one small table

Step 3.2: Bulk Load into Bronze Lakehouse (Data Engineering Responsibility)

⚠ Steps 3.2 and 3.3 are the Data Engineering team's deliverables, not yours. Your role is to provide requirements, validate outputs, and ensure the Silver layer is ready for your curated layer. You should understand these steps but not own them.

The DE team performs the one-time bulk migration of historical data:

1. In WS-Bronze, the DE team creates a **Data Pipeline** named PL_InitialLoad_SQLServer
2. CopyData activities load each source table from SQL Server into LH-Bronze
3. Configuration per activity:
   - **Source:** SQL Server connection > select table
   - **Destination:** LH-Bronze > Tables > table name matching the source
   - **Mode:** Overwrite (for initial load)
4. The DE team verifies row counts in the Bronze Lakehouse match the SQL Server source

Your involvement: Provide the DE team with the migration inventory from Phase 1 (which tables are needed, priority order, expected row counts). Review the loaded data after completion.

Step 3.3: Transform Bronze to Silver (Data Engineering Responsibility)

This is the Data Engineering team's responsibility. Your role is to collaborate on requirements and validate outputs:

- Meet with the Data Engineering team and provide a list of tables and transformations needed for the Silver layer
- Key transformations for Silver: deduplication, null handling, data type standardization, key conformance, soft-delete flagging
- The Data Engineering team implements these using Fabric Notebooks (PySpark) in WS-Silver
- **Your role:** validate that Silver layer tables have the correct schema, data quality, and completeness for your curated layer needs

```
-- Validate Silver table via the Lakehouse SQL analytics endpoint
SELECT COUNT(*) AS RowCount, COUNT(DISTINCT Booking_ID) AS UniqueBookings,
  SUM(CASE WHEN Gross_Sales IS NULL THEN 1 ELSE 0 END) AS NullGrossSales
FROM LH_Silver.dbo.Fact_Bookings
```

Do NOT proceed to Step 3.4 until Silver tables are validated and signed off.

Step 3.4: Build Gold Layer Dataflows (Your Responsibility Starts Here)

JD requirement: "Implement Dataflows, governed datasets." This is where your ownership begins. Create Dataflow Gen2 items that read from the Silver Lakehouse and write to the Gold Lakehouse:

⚠ CRITICAL: Gold Dataflows must read from the Silver Lakehouse SQL analytics endpoint — NEVER directly from SQL Server. The entire point of the medallion architecture is that each layer reads from the layer below it. SQL Server → Bronze → Silver → Gold. If the Gold layer connects directly to SQL Server, you bypass all cleansing, deduplication, and conformance done in Silver.

5. In WS-Gold-Curated-DEV, create a **Dataflow Gen2** named DF_Gold_Fact_Bookings

6. Click Get Data > Lakehouse — select the LH-Silver Lakehouse SQL analytics endpoint as your source (NOT SQL Server)
7. Select the Silver table you need (e.g., Silver_Fact_Bookings)
8. In Power Query Editor, apply Gold-layer transformation steps:
    - Join with dimension keys if needed
    - Calculate derived columns (e.g., Net_Revenue = Gross_Margin - Commission)
    - Apply business rules (e.g., exclude cancelled orders from revenue)
    - Rename columns to match the Gold layer naming convention
    - Apply data type conversions and formatting
9. **Set the Output Destination:** Click the gear icon on the query > select Lakehouse > LH-Gold > choose table name (e.g., Fact_Bookings)
10. **Set the Update Method:** Replace for dimension tables (full reload), Append for fact tables (incremental)
11. Repeat for each Gold table: Fact_Bookings, Fact_Operating_Expenses, Dim_Date, Dim_Brand, Dim_Region, Dim_Sales_Channel
12. Publish and test each Dataflow

Step 3.5: Migrate Existing Gen1 Dataflows to Gen2

If Gen1 Dataflows were identified in Phase 1:

- Open the Gen1 Dataflow in the Power BI Service
- Copy the Power Query (M) code from each query
- Create a new Dataflow Gen2 in WS-Gold-Curated-DEV
- Paste the M code into the Advanced Editor of the new Dataflow Gen2
- Change the output destination from Power BI dataset to Lakehouse table
- Test, validate, and publish

✓ CHECKPOINT: You should now have: Bronze and Silver Lakehouses loaded and validated (by the DE team with your input), Gold Dataflow Gen2 items created that read from Silver and write to Gold, and any Gen1 Dataflows migrated to Gen2. The critical validation: every Gold Dataflow reads from LH-Silver, never from SQL Server directly.

Phase 4: Build the Curated Gold Layer

JD: *"Develop curated semantic models and KPI logic on top of the Lakehouse architecture" + "Design business-ready views and metrics within the curated layer" + "Collaborate closely with the Data Engineering team, leveraging Bronze / Silver layers to build trusted business metrics in the curated layer"*

This is the core of your role. The Gold layer transforms Silver data into business-ready tables that consumers trust and use daily.

Step 4.1: Design the Gold Layer Schema

Open the LH-Gold Lakehouse SQL analytics endpoint and design your star schema:

- **Fact Tables:** Business events with measurable values (Fact_Bookings, Fact_Bookings, Fact_Transactions). Reference the Silver layer tables as your source.
- **Dimension Tables:** Descriptive attributes (Dim_Brand, Dim_Date, Dim_Region, Dim_Sales_Channel). These provide the filter/slicer context for reports.
- **Date Dimension:** Always create a dedicated Dim_Date table with Year, Month, Quarter, Fiscal Year, Fiscal Quarter, Day of Week, etc. This is essential for time intelligence in semantic models.

Step 4.2: Create Gold Tables Using Dataflow Gen2

For each Gold table, create a Dataflow Gen2 that reads from Silver and writes to Gold:

13. **Create Dataflow Gen2:** DF_Gold_Fact_Bookings
14. Get Data from the Silver Lakehouse SQL analytics endpoint
15. Apply Gold-layer transformations:
    - Join fact data with dimension keys
    - Calculate derived columns (e.g., Net_Revenue = Gross_Margin - Commission)
    - Apply business rules (e.g., exclude cancelled orders from revenue calculations)
    - Aggregate to the required grain (daily, monthly, per-transaction)
16. Set output destination to LH-Gold > Tables > Fact_Bookings
17. Set update method: Replace for dimension tables, Append for fact tables
18. Repeat for each Gold table

Step 4.3: Create Business-Ready SQL Views

JD: "Design business-ready views and metrics within the curated layer."

In the LH-Gold SQL analytics endpoint, create views that encapsulate business logic:

```
-- Example: Business-ready view for sales KPIs by month
CREATE VIEW vw_Monthly_Sales_KPIs AS
SELECT
  d.Year, d.Month_Name, d.Quarter,
  b.Brand,
  SUM(f.Gross_Sales) AS Gross_Sales,
  SUM(f.Gross_Margin) AS Gross_Margin,
  SUM(f.Gross_Margin) * 1.0 / NULLIF(SUM(f.Gross_Sales), 0) AS Margin_Pct,
  SUM(f.Commission) AS Commission,
  SUM(f.Net_Revenue) AS Net_Revenue,
  COUNT(*) AS Booking_Count
FROM Fact_Bookings f
  JOIN Dim_Date d ON f.Booking_Date = d.Date
  JOIN Dim_Brand b ON f.Brand = b.Brand
GROUP BY d.Year, d.Month_Name, d.Quarter, b.Brand
```

⚠ IMPORTANT: SQL views in the Lakehouse SQL analytics endpoint are NOT compatible with Direct Lake mode. Direct Lake reads only from Lakehouse tables. If you want Direct Lake on the semantic model, materialize these views as Gold tables (via Dataflow Gen2 or Notebook). Use views only for ad-hoc analysis or DirectQuery semantic models.

Step 4.4: Materialize Views as Gold Tables (for Direct Lake)

Since Direct Lake cannot read SQL views, you need to materialize your business logic as physical Gold tables. Use Dataflow Gen2 for this — consistent with the JD requirement to "implement Dataflows" and avoids the need to write Spark code (which is the DE team's domain):

19. Create a Dataflow Gen2 named **DF_Gold_Monthly_Sales_KPIs**
20. Click Get Data > Lakehouse > select LH-Gold SQL analytics endpoint
21. Write a SQL query in the native query editor that replicates the view logic:
```
-- Paste the same SQL as the view definition
SELECT d.Year, d.Month_Name, d.Quarter, b.Brand,
  SUM(f.Gross_Sales) AS Gross_Sales,
  SUM(f.Gross_Margin) AS Gross_Margin,
  SUM(f.Gross_Margin) * 1.0 / NULLIF(SUM(f.Gross_Sales), 0) AS Margin_Pct,
  COUNT(*) AS Booking_Count
FROM Fact_Bookings f
  JOIN Dim_Date d ON f.Booking_Date = d.Date
  JOIN Dim_Brand b ON f.Brand = b.Brand
GROUP BY d.Year, d.Month_Name, d.Quarter, b.Brand
```
22. **Set the Output Destination:** LH-Gold > Tables > Gold_Monthly_Sales_KPIs
23. **Set the Update Method:** Replace (full refresh each time)
24. Publish the Dataflow

The semantic model then connects to **Gold_Monthly_Sales_KPIs** (a physical table) using Direct Lake — no fallback to DirectQuery.

When to use a Notebook instead: Only if the transformation logic is too complex for Power Query (e.g., window functions, recursive CTEs, or heavy cross-table logic that Power Query cannot express). In that case, ask the DE team to create a scheduled notebook that materializes the table, and your Dataflow Gen2 reads the output.

Step 4.5: Define and Standardize KPIs

JD: "Build trusted business metrics." Create a KPI reference document that defines every metric:

- **KPI Name:** e.g., Gross Margin %
- **Definition:** Gross Margin divided by Gross Sales, expressed as a percentage
- **Formula (DAX):** DIVIDE([Gross Margin], [Gross Sales], 0)
- **Source Table:** Gold_Monthly_Sales_KPIs or Fact_Bookings
- **Owner:** BI Development team
- **Last Validated:** Date

Store this reference in a shared location (SharePoint, Git wiki, or as a Lakehouse table). This ensures every report uses the same KPI definitions — the "trusted" part of the JD.

Step 4.6: Implement Data Validation on Gold Tables

- Create SQL validation queries that run after each Gold table refresh:
```
-- Check for null revenue in fact table
SELECT COUNT(*) AS NullGrossSales FROM Fact_Bookings WHERE Gross_Sales IS NULL
-- Check referential integrity
SELECT COUNT(*) AS OrphanedKeys FROM Fact_Bookings f
  LEFT JOIN Dim_Brand b ON f.Brand = b.Brand
  WHERE b.Brand IS NULL
```
- Log validation results to a dq_validation_log table
- Set up Power Automate alerts if critical validations fail

Step 4.7: Create the Refresh Orchestration Pipeline

⚠ Without orchestration, your Gold Dataflows may run before Silver refreshes complete — resulting in Gold tables built on stale data. This is a silent, hard-to-debug problem that erodes trust in your metrics.

Create a master Data Pipeline that chains the entire refresh sequence in order:

25. In WS-Gold-Curated-DEV, create a **Data Pipeline** named PL_Master_Refresh_Gold
26. **Activity 1 — Wait for Silver:** Add a Web activity or custom polling activity that checks if the DE team's Silver pipeline/Dataflow has completed. Alternatively, coordinate with the DE team to trigger your pipeline via a webhook or scheduling dependency (e.g., your pipeline runs at 7 AM, their Silver refresh completes by 6:30 AM).
27. **Activity 2 — Refresh Gold Fact Tables:** Add Dataflow activities that trigger each Gold fact table Dataflow Gen2 (DF_Gold_Fact_Bookings, DF_Gold_Fact_Bookings, etc.). These run in parallel if they're independent.
28. **Activity 3 — Refresh Gold Dimension Tables:** Add Dataflow activities for dimension tables. These can run in parallel with fact tables or before them, depending on dependencies.
29. **Activity 4 — Materialize Aggregate Tables:** Trigger DF_Gold_Monthly_Sales_KPIs and any other materialized aggregate Dataflows. These MUST run AFTER fact/dimension refreshes because they aggregate Gold tables.
30. **Activity 5 — Run Data Validation:** Trigger a notebook or SQL script that runs the validation checks from Step 4.6 and logs results to dq_validation_log.
31. **Activity 6 (optional) — Refresh Semantic Model:** If using Import mode for any semantic models, trigger a dataset refresh. If using Direct Lake, this step is unnecessary — data is available immediately.

Set error handling: On each activity, configure failure paths. If Activity 2 fails, skip Activity 4 (don't materialize aggregates on partial data). Send a Power Automate alert on any failure.

Schedule: Set PL_Master_Refresh_Gold to run on a schedule (e.g., daily at 7 AM, after the DE team's Silver refresh window). This single pipeline is the only scheduled item — individual Dataflows should NOT have their own schedules, or you risk race conditions.

⚠ CRITICAL: Remove individual refresh schedules from Gold Dataflow Gen2 items. Only the master pipeline should trigger refreshes. If individual Dataflows also have their own schedules, they'll run at unpredictable times without waiting for Silver data.

✓ CHECKPOINT: You should now have: Gold Lakehouse with materialized fact and dimension tables, standardized KPI definitions documented, business-ready SQL views for ad-hoc querying, data validation running after each refresh, and a master orchestration pipeline that chains Silver-wait → Gold refresh → materialization → validation in the correct order.

Phase 5: Build Scalable Power BI Semantic Models

JD: *"Develop curated semantic models and KPI logic" + "Implement governed datasets and scalable Power BI semantic models"*

Step 5.1: Create a Semantic Model in Fabric

32. In WS-Gold-Curated-DEV, click New > Semantic Model
33. Select the LH-Gold Lakehouse as the data source
34. Choose the Gold tables to include: Fact_Bookings, Fact_Operating_Expenses, Fact_Forecast_Targets, Dim_Date, Dim_Brand, Dim_Region, Dim_Sales_Channel, Financial_Assumptions
35. **Connection mode: Direct Lake** is automatically selected when connecting to Lakehouse tables
36. Name the model: SM_Executive_Reporting (or per your naming convention)

Step 5.1b: Set Up Tabular Editor for Development

⚠ The Fabric web modeler is limited for serious DAX development — creating 30+ measures in a web form is slow and error-prone. Use Tabular Editor for all DAX work.

37. **Download Tabular Editor** (free version: Tabular Editor 2, or paid: Tabular Editor 3) from tabulareditor.com
38. In the Fabric web portal, go to Semantic Model Settings > copy the XMLA endpoint (e.g., powerbi://api.powerbi.com/v1.0/myorg/WS-Gold-Curated-DEV)
39. In Tabular Editor, click File > Open From DB > paste the XMLA endpoint > select SM_Executive_Reporting
40. You now have a full code editor with:
    - Bulk measure creation (create all 30+ measures with proper formatting)
    - IntelliSense for DAX formulas
    - Best Practice Analyzer (flags issues like missing descriptions, unused columns, bidirectional relationships)
    - TMDL export for Git version control
    - Copy/paste measures between models
41. **After editing in Tabular Editor, click Save (Ctrl+S) to push changes back to the Fabric semantic model.** Changes are live immediately.

Best practice: Use Tabular Editor for all DAX development, relationship configuration, and model optimization. Use the Fabric web portal only for quick visual checks and testing.

Step 5.2: Configure Relationships in the Model

Open the semantic model in the web modeler or Tabular Editor. First, note that Fact_Operating_Expenses, Fact_Forecast_Targets, and Financial_Assumptions use Year/Month columns instead of a Date column. Create a calculated column in each to enable relationships with Dim_Date:

```
-- In Power Query for each table, add: Date = #date([Year], [Month], 1)
```

This creates the first of each month as the date key. Then configure these relationships:

Relationships from Dim_Date:

- Dim_Date[Date] → Fact_Bookings[Booking_Date] (1:Many, Active)
- Dim_Date[Date] → Fact_Bookings[Travel_Date] (1:Many, Inactive — use USERELATIONSHIP when needed)
- Dim_Date[Date] → Fact_Operating_Expenses[Date] (1:Many, Active)
- Dim_Date[Date] → Fact_Forecast_Targets[Date] (1:Many, Active)
- Dim_Date[Date] → Financial_Assumptions[Date] (1:Many, Active)

Relationships from Dim_Brand:

- Dim_Brand[Brand] → Fact_Bookings[Brand] (1:Many)
- Dim_Brand[Brand] → Fact_Operating_Expenses[Brand] (1:Many)
- Dim_Brand[Brand] → Fact_Forecast_Targets[Brand] (1:Many)

- Dim_Brand[Brand] → Financial_Assumptions[Brand] (1:Many)

Relationships from other dimensions:

- Dim_Region[Region] → Fact_Bookings[Region] (1:Many)
- Dim_Sales_Channel[Sales_Channel] → Fact_Bookings[Sales_Channel] (1:Many)

All relationships should be single-direction cross-filtering (dimension filters fact). Mark Dim_Date as the Date Table for time intelligence. The model should have 9 tables with no orphaned or unrelated tables.

Step 5.3: Create DAX Measures

Add these DAX measures to the semantic model. Use Tabular Editor for efficient bulk creation. Organize into display folders.

Core KPIs (folder: Core KPIs):

```
Gross Sales = SUM(Fact_Bookings[Gross_Sales])
Gross Margin = SUM(Fact_Bookings[Gross_Margin])
Gross Margin % = DIVIDE([Gross Margin], [Gross Sales], 0)
Total Commission = SUM(Fact_Bookings[Commission])
Commission % = DIVIDE([Total Commission], [Gross Sales], 0)
Net Revenue = SUM(Fact_Bookings[Net_Revenue])
Booking Count = COUNTROWS(Fact_Bookings)
Avg Booking Value = DIVIDE([Gross Sales], [Booking Count], 0)
Cancellation Rate = DIVIDE(CALCULATE(COUNTROWS(Fact_Bookings), Fact_Bookings[Status] =
"Cancelled"), COUNTROWS(Fact_Bookings), 0)
```

Profitability (folder: Profitability):

```
Total OpEx = SUM(Fact_Operating_Expenses[Amount])
Operating Profit = [Net Revenue] - [Total OpEx]
Operating Margin % = DIVIDE([Operating Profit], [Gross Sales], 0)
Staff Cost = CALCULATE([Total OpEx], Fact_Operating_Expenses[Expense_Category] = "Staff &
Salaries")
Marketing Cost = CALCULATE([Total OpEx], Fact_Operating_Expenses[Expense_Category] = "Marketing &
Advertising")
```

Time Intelligence (folder: Time Intelligence):

```
Gross Sales PY = CALCULATE([Gross Sales], SAMEPERIODLASTYEAR(Dim_Date[Date]))
YoY Growth % = DIVIDE([Gross Sales] - [Gross Sales PY], [Gross Sales PY], 0)
Gross Sales YTD = TOTALYTD([Gross Sales], Dim_Date[Date])
Gross Sales MTD = TOTALMTD([Gross Sales], Dim_Date[Date])
Gross Sales QTD = TOTALQTD([Gross Sales], Dim_Date[Date])
Gross Margin PY = CALCULATE([Gross Margin], SAMEPERIODLASTYEAR(Dim_Date[Date]))
Operating Profit PY = CALCULATE([Operating Profit], SAMEPERIODLASTYEAR(Dim_Date[Date]))
Operating Profit YoY % = DIVIDE([Operating Profit] - [Operating Profit PY], [Operating Profit PY],
0)
Gross Sales by Travel Date = CALCULATE([Gross Sales], USERELATIONSHIP(Dim_Date[Date],
Fact_Bookings[Travel_Date]))
```

Forecasting (folder: Forecasting):

```
Forecast Gross Sales = SUM(Fact_Forecast_Targets[Forecast_Gross_Sales])
Forecast Gross Margin = SUM(Fact_Forecast_Targets[Forecast_Gross_Margin])
Forecast Bookings = SUM(Fact_Forecast_Targets[Forecast_Bookings])
Prior Year Gross Sales = SUM(Fact_Forecast_Targets[Prior_Year_Gross_Sales])
Forecast Variance $ = [Gross Sales] - [Forecast Gross Sales]
Forecast Variance % = DIVIDE([Forecast Variance $], [Forecast Gross Sales], 0)
```

Financial Assumptions (folder: Assumptions):

```
Target Margin % = AVERAGE(Financial_Assumptions[Gross_Margin_Target_Pct])
Target Revenue Growth % = AVERAGE(Financial_Assumptions[Revenue_Growth_Target_Pct])
Budget Marketing = SUM(Financial_Assumptions[Marketing_Budget])
Assumed Cancellation Rate = AVERAGE(Financial_Assumptions[Cancellation_Rate_Assumption])
```

```
Margin vs Target = [Gross Margin %] - [Target Margin %]
Revenue vs Growth Target = [YoY Growth %] - [Target Revenue Growth %]
```

Total: 35 measures. Every measure references actual column names from your datasets. No translation needed.

Step 5.4: Implement Row-Level Security

- In the semantic model editor, go to Manage Roles
- Create a role for each security group that needs filtered data:
  - **Regional Manager:** DAX filter: Dim_Region[Region] = USERPRINCIPALNAME() — or map via a security table
  - **Brand Manager:** DAX filter: Dim_Brand[Brand] = "Luxury Tours"
  - **Executive:** No filter (sees all data)
- Test each role using "View as role" in Power BI Service
- Assign Entra ID security groups to roles in the dataset security settings

Step 5.5: Optimize the Semantic Model

JD: "Optimized for performance."

- Remove columns not used in reports or measures (reduces model size and improves Direct Lake performance)
- Hide foreign key columns from the Report view (users should filter through dimension attributes, not keys)
- Create display folders to organize measures logically (Core KPIs, Time Intelligence, Profitability, etc.)
- Set data categories on geography columns (Country, State, City) for map visuals
- Set default summarization to "Don't summarize" for ID and text columns
- Use DAX variables (VAR) in complex measures to avoid repeated calculations
- For Direct Lake: monitor fallback behavior in the Fabric capacity metrics app — if the model frequently falls back to DirectQuery, investigate the cause (typically unsupported features or model size exceeding memory)

Step 5.6: Create Additional Governed Datasets

JD: "Governed datasets." The executive reporting model may not serve every use case. Create additional semantic models for specific audiences:

- **SM_Executive_Reporting** — portfolio-level KPIs, brand performance, forecasting (for C-suite)
- **SM_Operational_Dashboard** — day-to-day metrics like booking volume, cancellation rates, channel performance (for operations)
- **SM_SelfService_Analytics** — governed dataset with Build permission for analysts to create their own reports

Each model connects to the same Gold Lakehouse tables but exposes different measures, hierarchies, and security rules.

✓ CHECKPOINT: You should now have: one or more semantic models in Direct Lake mode, connected to Gold Lakehouse tables, with DAX measures, RLS, and performance optimization applied.

Phase 6: Build Power BI Reports and Deliver

***JD:** "Contribute to a scalable analytics environment supporting executive reporting, operational dashboards, and advanced analytics"*

Step 6.1: Build Reports Using Live Connection

42. In Power BI Desktop or Power BI Service web authoring, create a new report
43. Choose "Connect to a Power BI semantic model" (not Get Data)
44. Select SM_Executive_Reporting from the WS-Gold-Curated-DEV workspace
45. Build report visuals using the measures and dimensions from the semantic model

Key principle: Reports contain ZERO data and ZERO DAX. All logic lives in the semantic model. Reports are pure visualization layers. This makes reports lightweight, reusable, and independent of the underlying data.

Step 6.2: Design Executive Dashboard Pages

Follow the same 4-page structure proven in the Kensington project:

- **Page 1 — Executive Summary:** KPI cards (gross sales, margin, profit, YoY), brand comparison chart, monthly trend
- **Page 2 — Drill-Through by Segment:** P&L waterfall, OpEx breakdown, regional performance for a selected brand/department
- **Page 3 — Forecast & Targets:** Actual vs. forecast, variance analysis, financial assumptions table
- **Page 4 — Operational Detail:** Channel mix, booking funnel, destination/product detail

Step 6.3: Configure Deployment Pipeline Rules (Critical)

⚠ Without deployment pipeline rules, reports promoted from DEV to PROD will still point to the DEV semantic model — meaning production reports query DEV data. This is a common and dangerous mistake.

Before deploying any reports, configure dataset rebinding rules:

46. In the Fabric Deployment Pipeline, click the gear icon on the TEST stage
47. Go to Deployment Rules > Dataset Rules
48. **Create a rule:** Map SM_Executive_Reporting (DEV) → SM_Executive_Reporting (TEST). This tells the pipeline: when a report is promoted from DEV to TEST, rebind it to the TEST version of the semantic model.
49. Repeat for the PROD stage: Map SM_Executive_Reporting (TEST) → SM_Executive_Reporting (PROD)
50. If you have multiple semantic models (SM_Operational_Dashboard, SM_SelfService_Analytics), create rules for each one
51. Save the rules

Also configure parameter rules if your Dataflows or pipelines use parameterized connections (e.g., a Lakehouse parameter that differs between DEV and PROD). The pipeline can swap parameter values during deployment.

Step 6.4: Deploy Reports Through the Pipeline

52. First, deploy the semantic model(s) from DEV → TEST → PROD using the Deployment Pipeline. The semantic model must exist in each stage before reports can bind to it.
53. Then, deploy reports from DEV → TEST. The pipeline auto-rebinds the report to the TEST semantic model (per the rules you configured).
54. Stakeholders validate the reports in TEST against TEST data.
55. Deploy TEST → PROD. Reports rebind to the PROD semantic model.
56. Verify all visuals render correctly with production data.

Deployment order matters: Always deploy semantic models first, then reports. If you deploy a report before its target semantic model exists in that stage, deployment will fail.

Step 6.5: Create and Publish the Power BI App

57. In WS-Reporting-PROD, click "Create App"
58. Name: "[Company] Executive Reporting"
59. Configure navigation: arrange pages in logical order (Executive Summary first)
60. Set audience: assign SG-Fabric-ReportConsumers and SG-Fabric-Executives
61. Publish the App

- Users access the App from the Power BI Apps section — one clean entry point with curated navigation

Step 6.6: Enable Self-Service Analytics

JD: "Self-service analytics."

- On the SM_SelfService_Analytics semantic model, grant Build permission to SG-Fabric-SelfServiceAnalysts
- Self-service analysts can now create their own reports in Power BI Desktop or web using the governed dataset
- They get all the curated measures, standardized KPIs, and RLS — but can build whatever visuals they need
- They CANNOT modify the underlying semantic model or access raw data

This is the balance between governance and flexibility that the JD requires.

✓ CHECKPOINT: You should now have: published reports in the Reporting workspace, a Power BI App available to consumers, and self-service analytics enabled via governed datasets with Build permission.

Phase 7: Establish Governance and Document Everything

***JD:** "Help establish best practices for BI development and data governance" + "Ensure datasets are documented, reusable, and optimized for performance"*

Step 7.1: Document Every Governed Dataset

For each semantic model and Gold table, create a metadata record:

- **Dataset Name:** SM_Executive_Reporting
- **Description:** Portfolio-level performance, profitability, and forecasting across all brands
- **Source:** LH-Gold tables (Fact_Bookings, Dim_Date, Dim_Brand, etc.)
- **Connection Mode:** Direct Lake
- **Refresh Schedule:** Automatic via Direct Lake (no scheduled refresh needed)
- **RLS:** Yes — Regional Manager, Brand Manager, Executive roles
- **Measures:** 32 DAX measures across Core KPIs, Time Intelligence, Profitability folders
- **Owner:** BI Development team
- **Endorsed:** Yes (Promoted or Certified in Power BI Service)

Step 7.2: Endorse Datasets in Power BI Service

- In Power BI Service, go to each governed semantic model's settings
- **Endorsement:** Set to "Promoted" for validated datasets, or "Certified" for datasets that have passed formal review
- Endorsed datasets appear with a badge in the Power BI data hub, making it easy for self-service analysts to find the trusted source
- Add a description and contact info so consumers know who to reach out to

Step 7.3: Create a BI Development Standards Document

Document these best practices for the team:

- **Naming conventions:** LH_ for Lakehouses, DF_ for Dataflows, SM_ for Semantic Models, PL_ for Pipelines, RPT_ for Reports
- **Git workflow:** feature branches for new development, pull requests for review, merge to main for deployment
- **Deployment pipeline rules:** no direct edits in PROD — all changes flow through DEV → TEST → PROD
- **DAX standards:** all measures in a dedicated _Measures table (or display folder), use VAR for readability, include comments for complex logic
- **Semantic model standards:** hide foreign keys, set data categories, organize measures in folders, remove unused columns
- **Testing protocol:** every new measure must be validated against known-good values before promotion to PROD
- **Refresh monitoring:** check Dataflow Gen2 refresh history weekly, investigate any failures

Step 7.4: Build a Data Quality Dashboard

- Create a Power BI report connected to the dq_validation_log table (created in Phase 4)
- Show: validation pass/fail rates over time, datasets with most failures, most common failure types
- This is an operational dashboard in its own right — part of the "operational dashboards" the JD calls for

Step 7.5: Plan for Future AI Capabilities

JD: "Future AI capabilities." The Fabric environment you've built is AI-ready because:

- **Lakehouse:** stores data in Delta format which is directly accessible by Fabric's AI features (ML models, Copilot, etc.)

- **Gold tables:** curated, validated, and documented — exactly what AI models need as training data
- **Semantic models:** Copilot for Power BI can use semantic model metadata (measure names, descriptions) to generate natural language insights
- **Capacity:** Fabric capacity includes AI/ML compute that can be enabled when needed

When the organization is ready for AI, the data infrastructure is already in place. No re-architecture needed.

Step 7.6: Decommission Legacy SQL Server / Power BI Desktop Sources

Once all consumers are migrated to Fabric:

62. Verify all reports point to Fabric semantic models (no lingering SQL Server direct connections)
63. Archive old .pbix files and Gen1 Dataflows
64. Work with IT to decommission the SQL Server instance or downgrade it if still needed for other purposes
65. Update any documentation that references the old environment
66. Communicate the switch to stakeholders with a clear "what changed and where to find things" guide

✓ CHECKPOINT: You should now have: all datasets documented and endorsed, BI development standards published, data quality monitoring in place, legacy sources decommissioned, and the Fabric environment positioned for future AI capabilities.

End of Guide